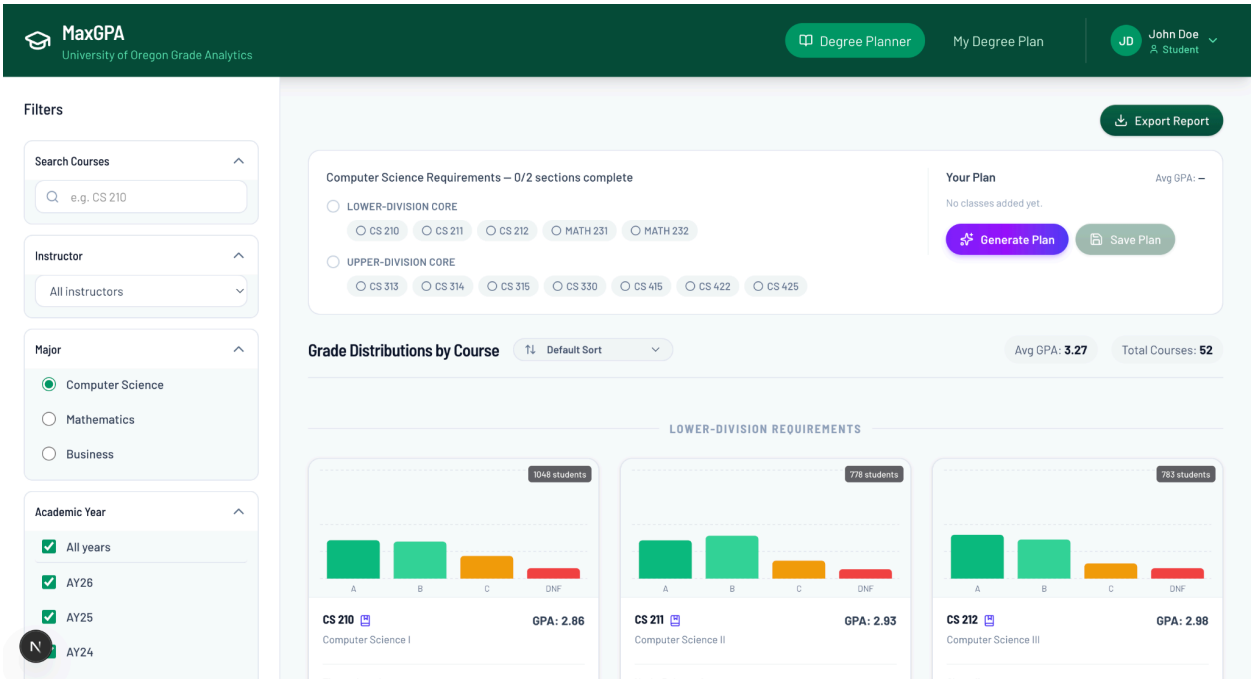
Group 7 Final Submission



1

Authors:

Kaegan Koski, Robert Mimms, Hiro Nakae, Peyton Phillips

Date:

05/04/2026

[1] Finished product image

Table of Contents

Project Plan

Management

I. People
   A. Peyton Phillips
      1. FastAPI
         a) FastAPI is used for getting stat files from backend for use with other libraries
         b) GET /api/major/course/{course_code}: returns the average grade for that course
         c) GET /api/major/instructor/{instuctor_id}: returns average grades given by that teacher.
      2. Creates visualization for data using PANDAS and NumPy libraries
         a) PANDAS is used for parsing CSV files that are gotten from backend, as well as putting the data into graph format
         b) NumPy is used for any math functions that could be needed (linear algebra, vector multiplication, etc.)
   B. Robert Mimms
      1. SQLite3 Database + Queryable data
      2. Turn the CSV into a schema design
      3. Write script to parse CSV
      4. Seed database
   C. Kaegan Koski
      1. Testing of code
         a) Making sure all parts work together
         b) Integrating parts together if they don't
      2. Documentation
         a) Takes notes, writes down general goals to keep (plans)
         b) Actively changes plan if needed, otherwise just monitors
   D. Hiro Nakae
      1. Design
         a) Student and Admin Pages.
         b) Fetching and displaying data on the graphs.
      2. Implementation

a) Web(client) pages, client-side api calls, chart library integration.
b) Use a Next.js Server Component to fetch() that data.
c) Pass that JSON to a Client Component where Chart.js renders the bar graph.

3. Testing
    a) Use end-to-end testing to verify dynamic ui displays correct data.
4. Documentation

II. Structure

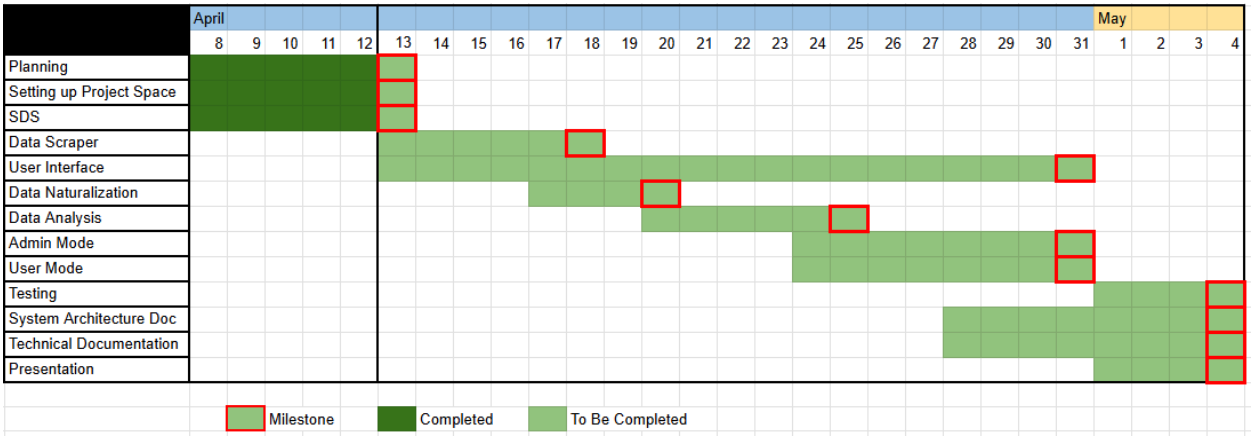
A. Roles

1. At time of writing, a ‘manager’ role doesn’t seem needed. People are willing to fill in the role as needed later on though.
2. Mainly, people will work in their specifications described above. However, if a part of the project falls behind the Gantt chart, others can step in to work with the person in that area until caught up

B. Team based

1. We meet in-person weekly at minimum, most likely 2 meeting per week
2. Communication with each other (when not in person) is done over text
3. Meetings are 1 hour long
4. Mandatory reports are done by Peyton

Work Schedule



[2]

- Key

[2] Gantt Chart of the schedule

    - P → Peyton
    - K→ Kaegan
    - R → Robert
    - H → Hiro
    - G → Group / All
- Milestones
    - Planning (G)
    - Setting up Project Space (G)
    - SDS (G)
    - Data Scraper (R)
    - User Interface (H)
    - Data Naturalization (P)
    - Data Analysis (P)
    - Admin Mode (H)
    - User Mode (H)
    - Testing (K)
    - System Architecture Documentation (K)
    - Technical Documentation (K)
    - Presentation (G)

Monitoring + Reporting

- Github commits will be reviewed weekly to track code contributions
- Task_And_Assignment_Breakdown will track:
    - Current status on assigned work
    - Any changes to expected timeline outside of the Gantt chart
- Weekly meeting notes will include:
    - Attendees
    - Decisions made
    - Completed tasks
    - Blockers

Build Plan

The planning phase is a significant portion of the schedule so as to highlight the importance of this step. The first project deliverables can all be worked on in parallel, so they are indicated as such.

As soon as the planning phase is completed, work can begin on the database. Once the data can be fully loaded into the system, the API layer can be implemented which includes naturalization and parts of analysis. The rest of analysis includes generation of the distribution tables which can only be made when the API layer is complete. These tasks all lay on the critical path.

Once these systems are implemented, the user and admin mode can be implemented using the database, API layer, and distribution generation. Testing can be done in parallel with most of the things on this list.

The user interface, while reliant on all of these systems, can be designed and prepared to a large extent even before each of these subsystems are complete. This is indicated by the user interface being parallel to every other milestone on the list.

Finally, the implementation of documentation does require that the system is stable and finished in design. The presentation and documentation will become the main focus of the final days of the project.

Risk and Risk Mitigation:

1. Inconsistency in the CSV data files
    - Risk: malformed or missing data could break ingestion
    - Mitigation: implement validation checks before database verification
2. API performance bottlenecks
    - Risk: slow queries on large datasets
    - Mitigation: optimize queries and precompute aggregates
3. Integration issues between frontend and backend
    - Risk: mismatched data formats
    - Mitigation: define strict API contracts early
4. Team coordination delays
    - Risk: dependencies blocking progress
    - Mitigation: weekly check ins + a shared task tracker

SDS

Revisions History

Date	Author	Description
4/15/2026	Group 7	Initial submission
4/29/2026	Group 7	Final submission created
5/05/2026	Group 7	Reformatted to match SDS template
5/05/2026	Group 7	Updated design description to reflect the implemented Next.js, FastAPI, SQLite, and localStorage architecture

System Overview

MaxGPA is a web application that helps University of Oregon students compare courses and generate degree plans using historical grade distribution data. The system allows a student to filter courses by major, academic year, course search text, and instructor. It displays grade distributions, average GPA values, course recommendations, and instructor-specific grade information.

The system also includes an admin mode for importing grade-distribution CSV files and configuring degree requirements by major. Admin users can upload CSV files for grade data, preview the first valid row before importing, upload degree-plan requirement CSVs, add or remove requirement groups, and add or remove courses from requirement groups.

The implemented system is organized into a Next.js frontend, a FastAPI backend, a SQLite database, CSV ingestion utilities, requirement-management logic, degree-plan generation logic, and browser-side saved-plan storage. The frontend communicates with the backend through /api/* requests, which are rewritten by the Next.js configuration to the FastAPI server running on localhost:8000.

The system currently supports three major keys (CS for computer science, MATH for mathematics, and BA for business administration).

Software Architecture

Architecture overview

The MaxGPA system is organized into three major software layers: the Database Layer, the API Layer, and the UI Layer. This organization matches the implemented system structure and separates the responsibilities of data storage, backend service logic, and user interaction.

The Database Layer stores and imports academic data. The API Layer provides controlled access to that data through HTTP endpoints. The UI Layer displays the user interface and manages student/admin interactions.

Software Modules

Database Layer

Role and Primary Function

The Database Layer is responsible for persistent storage, database initialization, CSV ingestion, grade normalization, course-title storage, major requirement storage, and requirement-group storage.

This layer is implemented primarily through:

- db/database.py
- db/grade_data.db
- data/raw/*.csv
- data/meta/Reconciliation.csv

The Database Layer stores historical grade data and degree requirement data in SQLite. It also contains the logic for preparing raw CSV data before it enters the database. Its main responsibilities include:

- Creating SQLite tables if they do not already exist.
- Creating database indexes for common query patterns.
- Reading raw grade-distribution CSV files.
- Cleaning and normalizing grade data.
- Removing redacted or invalid grade rows.
- Aggregating plus/minus grades into base grade buckets.
- Storing course titles.
- Storing major requirement groups.
- Storing individual major course requirements.
- Loading reconciliation data for normalized course or title changes.

The grade data is normalized into four main buckets:

- Grade_A = A+ / A / A-
- Grade_B = B+ / B / B-
- Grade_C = C+ / C / C-
- Grade_DNF = D+ / D / D- / F / N

This normalization makes the frontend charts and backend GPA calculations consistent across all courses.

Interface to Other Layers

The Database Layer does not communicate directly with the browser. It is accessed by the API Layer through Python database functions and SQL queries. The API Layer uses the Database Layer to:

- Query courses.
- Query instructors.
- Query academic years.
- Query subjects.
- Query major requirements.
- Insert or delete requirement groups.
- Insert or delete course requirements.
- Preview and import grade CSV files.
- Preview and import degree requirement CSV files.

Important database/import functions include:

- initialize_database(data_dir)
- clean_and_aggregate_data(df)
- preview_grade_csv(file_obj)
- import_grade_csv(file_obj, db_path)
- preview_degree_plan_csv(file_obj)
- import_degree_plan_csv(file_obj, major_key, db_path)
- load_reconciliation(path)
- apply_reconciliation(df, title_map, course_num_map)

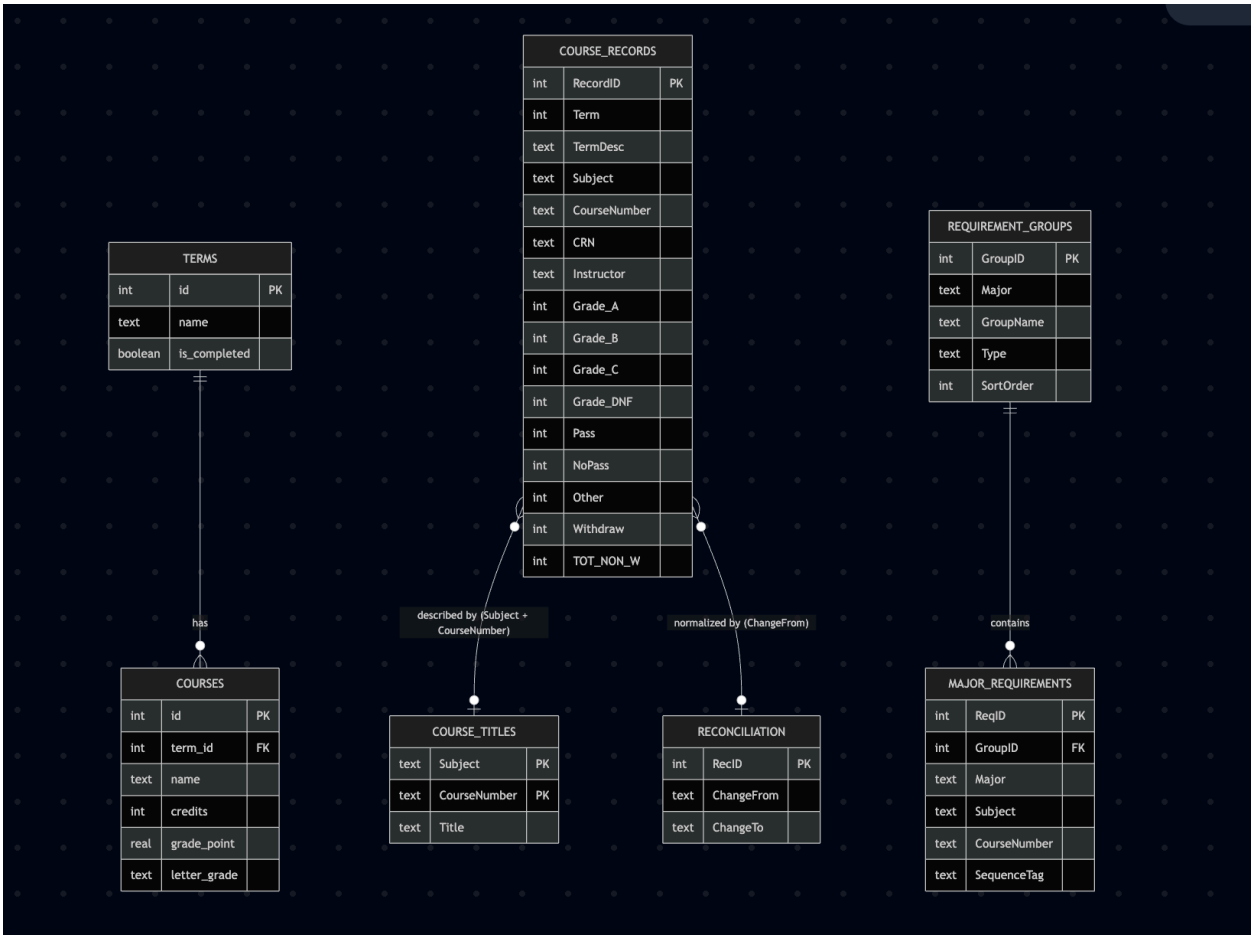
The main database tables are:

- Course_Records
- Course_Titles
- Requirement_Groups
- Major_Requirements

- Reconciliation

The Database Layer returns raw rows, grouped records, or import summaries to the API Layer. The API Layer is then responsible for converting those results into JSON responses for the frontend.

Static Model



[3]

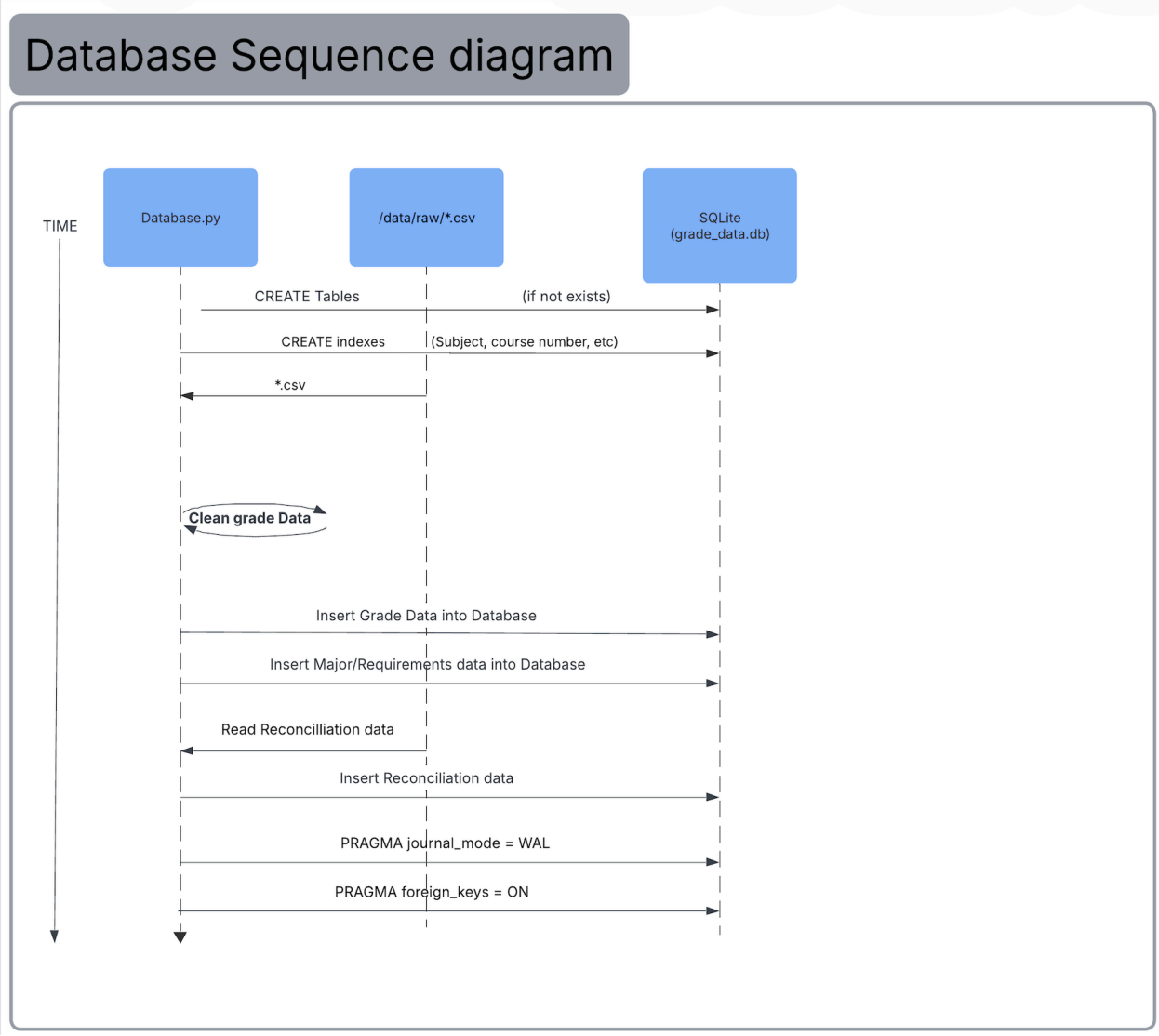
The static database model shows the structure of the persistent data used by MaxGPA. The most important table is Course_Records, which stores historical grade records by term, subject, course number, CRN, instructor, and normalized grade counts. Course_Titles stores human-readable course titles, while Requirement_Groups and Major_Requirements represent degree requirements for supported majors.

The relationship between Requirement_Groups and Major_Requirements allows a major to contain multiple groups of requirements. Each group can then contain multiple course

[3] Entity-relationship diagram for the SQLite database, showing course records, course titles, reconciliation data, requirement groups, and major requirements.

requirements. This structure supports requirement categories such as lower-division courses, upper-division courses, sequences, and elective groups.

Dynamic Model



[4]

The dynamic database model shows how the database is initialized and populated. During initialization, database.py creates the required tables and indexes. It then reads raw CSV files from the data directory, processes each row, normalizes grade buckets, and inserts cleaned records into SQLite.

The import process also inserts course-title data and loads reconciliation data. SQLite settings such as write-ahead logging and foreign key enforcement are configured during initialization to improve reliability and consistency.

[4] Database Sequence Diagram

Design Rationale

The Database Layer is separated from the API and frontend layers so that data storage and data preparation can be maintained independently from user interface behavior. This separation prevents frontend code from needing to know table names, SQL queries, or CSV-cleaning rules.

SQLite was selected because the project uses a local academic dataset and does not require a separate database server. This keeps setup simpler while still supporting structured queries, indexes, and persistent data storage.

The team chose to normalize grade data during import rather than during every API request. This reduces the repeated processing and ensures that all course, instructor, and GPA calculations use the same grade categories. The tradeoff is that import logic must be carefully maintained, because any mistake in the normalization step affects later API results.

The database schema separates grade records from degree requirements. This is important because historical grade data and curriculum requirements are different kidneys of information and may be updated independently.

API Layer

Role and Primary Function

The API Layer is responsible for communication between the Next.js frontend and the SQLite database. It receives HTTP requests from the frontend, validates request data, queries the database layer, performs backend calculations, and returns JSON responses. This layer is implemented primarily through:

- Course search and filtering
- Instructor lookup
- Course-specific instructor rankings
- Bulk best-instructor lookup
- Academic year lookup
- Subject lookup
- Requirement group lookup
- Requirement group creation
- Requirement group deletion
- Course requirement creation
- Course requirement deletion
- Grade CSV preview and upload
- Degree requirement CSV preview and upload

The API Layer also performs calculations needed by the frontend, including:

- Aggregating grade counts
- Computing average GPA
- Computing grade percentages
- Grouping requirements by major and major requirement group
- Selecting instructor analytics for courses

Interface to Other Layers

The API Layer receives requests from the UI Layer through /api/* paths. In the implemented system, Next.js rewrites frontend /api/* requests to the FastAPI server.

Current API routes include:

- GET /api/requirements
- POST /api/requirement-groups
- DELETE /api/requirement-groups/{group_id}
- POST /api/requirements
- DELETE /api/requirements/{req_id}

- POST /api/preview-degree-plan
- POST /api/upload-degree-plan

- POST /api/preview-csv
- POST /api/upload-csv

- GET /api/instructors
- GET /api/course-instructors
- GET /api/bulk-best-instructors
- GET /api/subjects
- GET /api/academic-years
- GET /api/courses

The API Layer communicates downward with the Database Layer through SQL queries and database helper functions.

Example request from the UI layer:

- GET /api/courses?subjects=CS,MATH&years=AY24,AY25

Example response shape:

```
[
  {
    "code": "CS 210",
    "name": "Computer Science I",
    "avgGpa": 3.24,
    "totalStudents": 184,
    "gradeData": [
      { "grade": "A", "count": 80, "percentage": 43.5 },
      { "grade": "B", "count": 55, "percentage": 29.9 },
      { "grade": "C", "count": 30, "percentage": 16.3 },
      { "grade": "DNF", "count": 19, "percentage": 10.3 }
    ]
  }
]
```

The API Layer also accepts request bodies for admin edits. For example, requirement group creation uses a request model similar to:

GroupIn

- major
- groupName
- type
- sortOrder

Course requirement creation uses a request model similar to:

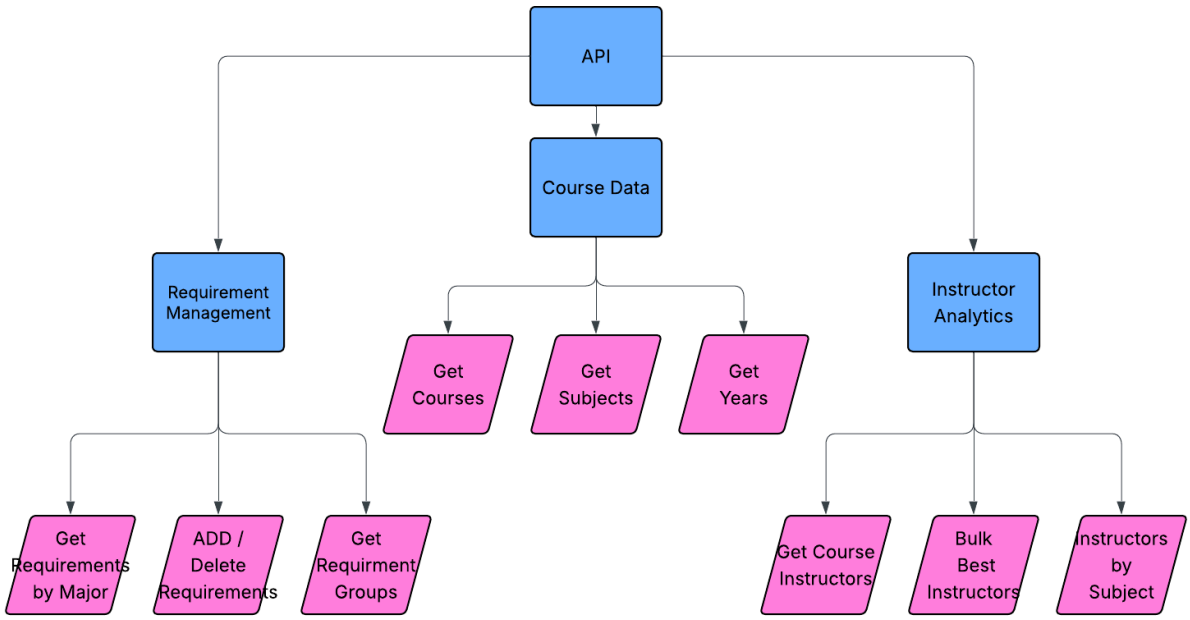
RequirementIn

- major
- groupId
- subject
- courseNumber
- sequenceTag

Static Model

[5]

The endpoint map shows how the API Layer is grouped by responsibility. Requirement-management endpoints support the admin portal and degree planning logic. Course-data endpoints support the student dashboard and course filtering. Instructor-analytics endpoints support instructor comparison and degree plan generation.



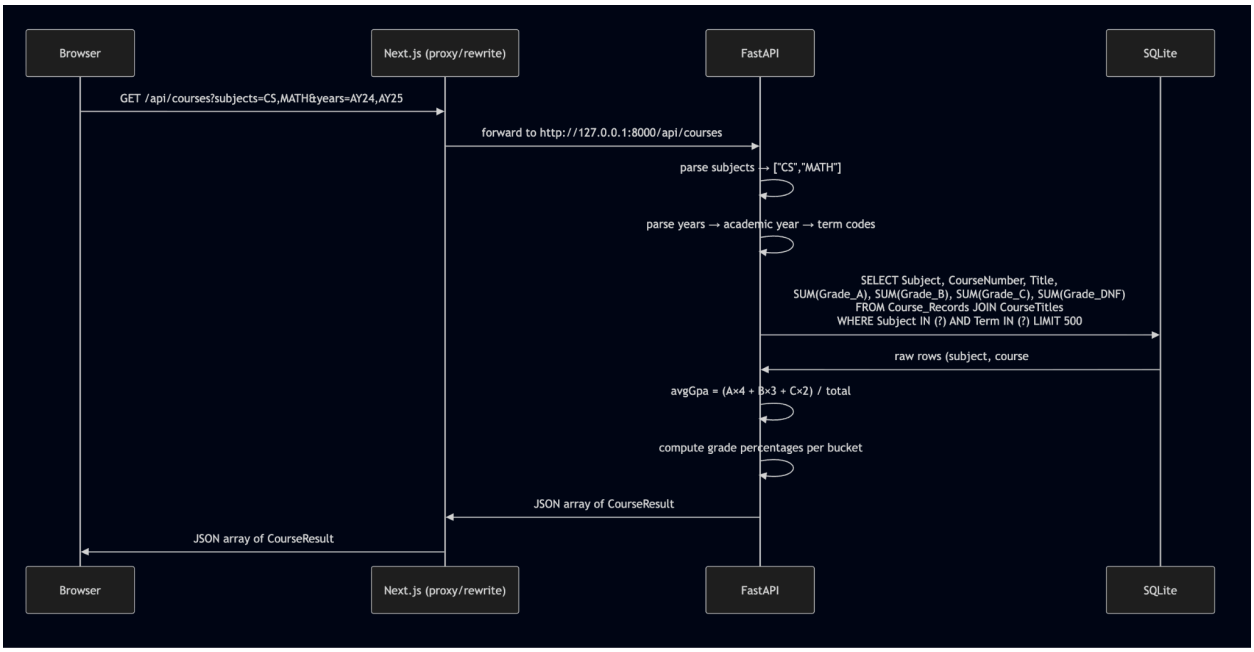
[6]

The API data model diagram shows the main shapes of data exchanged across the API. GroupIn and RequirementIn represent incoming admin requests. RequirementGroup and RequirementCourse represent requirement data returned to the frontend. CourseResult, InstructorResult, and GradeEntry represent course and instructor analytics returned to the student dashboard.

---

[5] FastAPI Endpoint Map
[6] API Data Model Class Diagram

Dynamic Model



[7]

The dynamic API model shows how a course request travels through the system. The browser sends a request to /api/courses, and Next.js forwards it to FastAPI. FastAPI parses the selected subjects and academic years, converts academic-year labels into term codes, queries SQLite, and aggregates grade counts. It then calculates average GPA and grade percentages before returning a JSON array of course results to the frontend.

Design Rationale

The API Layer provides a controlled boundary between the frontend and database. This prevents the frontend from directly depending on SQL queries, database tables, or data-cleaning implementation details.

FastAPI was chosen because the backend is written in Python and needs to work closely with pandas, SQLite, and CSV import functions. FastAPI also supports clear route definitions and request models, which makes admin operations such as adding requirement groups or uploading CSV files easier to structure.

The API Layer performs aggregations and GPA calculations before sending data to the frontend. This keeps the frontend focused on display and interaction rather than database-specific calculations. It also makes results more consistent because all frontend views receive data from the same backend calculation logic.

[7] Get Courses Request Sequence Diagram

The User Interface rewrite/proxy design allows frontend code to call /api/* paths without hardcoding the backend port throughout the application. This makes the frontend code cleaner and keeps backend routing details centralized in configuration.

User Interface Layer

Role and Primary Function

The User Interface Layer is the browser-facing layer of MaxGPA and is implemented with Next.js and React components. It includes the student dashboard, admin portal, saed plans page, shared layout, navigation header, filters, course cards, charts, requirement checklist, and plan export behavior.

This layer is implemented through the Next.js application files, including:

- app/page.tsx
- app/admin/page.tsx
- app/saved-plans/page.tsx
- app/components/app-layout.tsx
- app/components/filter-sidebar.tsx
- app/components/course-card.tsx
- app/components/grade-distribution-chart.tsx
- app/components/requirements-checklist.tsx
- app/components/csv-upload-zone.tsx
- app/lib/requirements.ts
- app/lib/saved-plans.ts

The UI Layer supports two main user modes:

1. Student mode, where users filter courses, view grade distributions, generate degree plans, save plans, and export plans

2. Admin mode, where users upload grade data, upload requirement data, and edit requirement groups.

The UI Layer also stores saved plans and draft plans in browser localStorage.

Interface to Other Layers

The Next.js Layer communicates upward with the user through browser UI components and downward with the API Layer through fetch requests to /api/*. Student-facing API calls include:

- GET /api/requirements
- GET /api/instructors
- GET /api/courses
- GET /api/course-instructors
- GET /api/bulk-best-instructors

Admin-facing API calls include:

- POST /api/preview-csv
- POST /api/upload-csv
- POST /api/preview-degree-plan
- POST /api/upload-degree-plan
- POST /api/requirement-groups
- DELETE /api/requirement-groups/{group_id}
- POST /api/requirements
- DELETE /api/requirements/{req_id}

The UI Layer also uses browser localStorage for saved plans. Important saved-plan helper functions include:

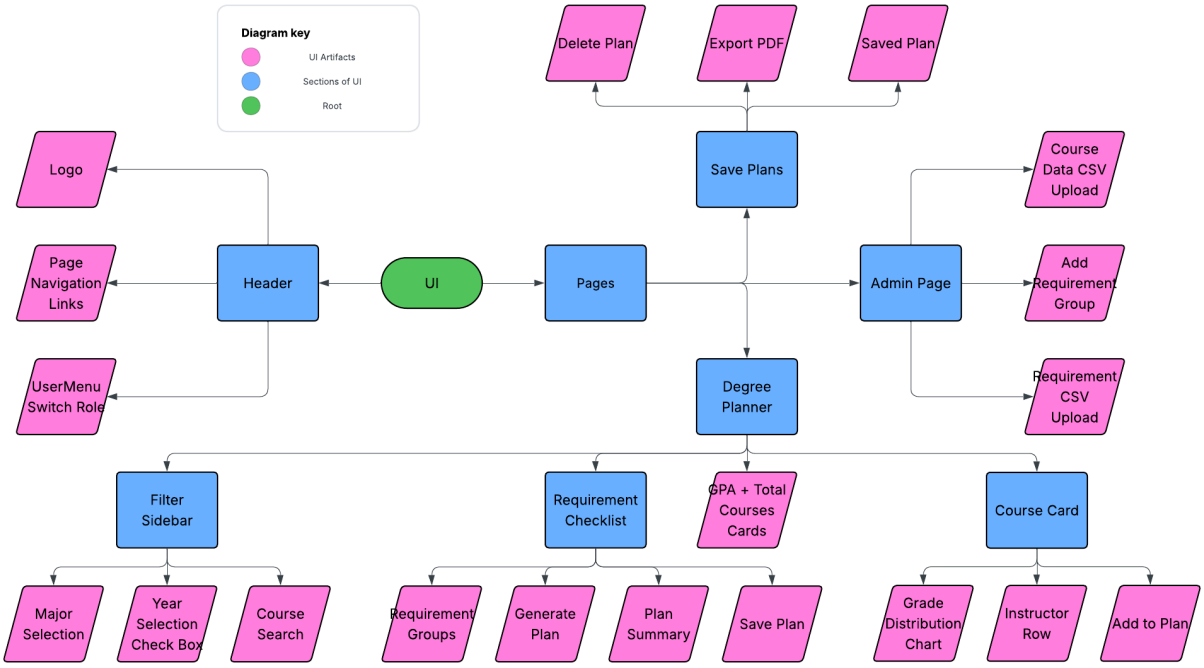
- getSavedPlans()
- savePlan(plan)
- deletePlan(id)
- getDraftItems(subject)
- addDraftItem(item, subject)
- removeDraftItem(code, instructor, subject)
- clearDraft(subject)

PDF export uses frontend libraries:

- html-to-image
- jsPDF

The UI Layer receives API data and renders it using React components. For example, course data is passed into course cards and then into Recharts-based grade distribution charts.

Static Model



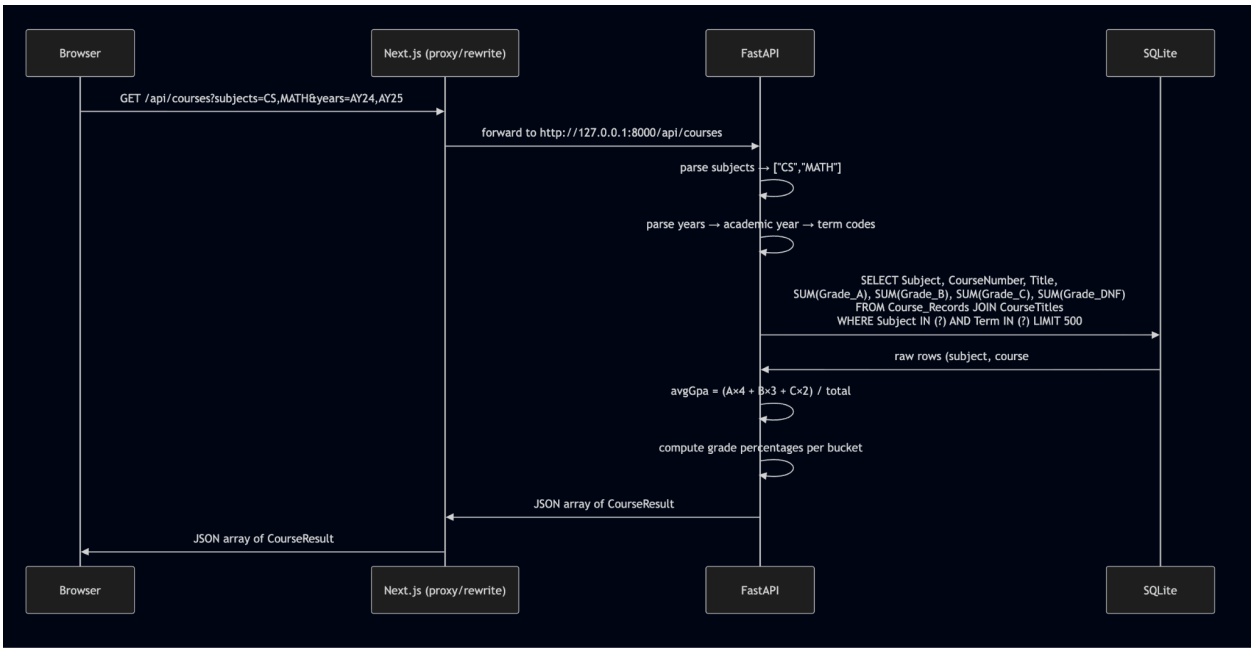
[8]

The static frontend model shows that AppLayout contains the shared header and navigation elements. The main pages include the degree planner page, saved plans page, and admin portal. The student degree planner page contains the filter sidebar, requirement checklist, GPA card, course cards, grade distribution charts, instructor rows, and plan-saving controls. The admin portal contains CSV upload components and requirement group editing controls.

This hierarchy separates common layout behavior from page-specific behavior. It also separates smaller reusable UI components, such as course cards and charts, from larger page-level components.

[8] Application Component Hierarchy

Dynamic Model



9

Design Rationale

The UI Layer is separated from the API and Database layers so that user interface behavior can change without modifying backend storage or SQL logic. This separation also keeps sensitive data operations, such as database updates and CSV imports, out of the browser.

The frontend is organized around reusable components. The filter sidebar, course card, chart, requirement checklist, upload zone, and saved plan card each handle a specific part of the interface. This makes the UI easier to maintain and reduces duplication.

Charts are rendered in the browser using Recharts because the API already returns chart-ready grade data. Rendering charts on the frontend avoids the need to generate and store chart images on the backend.

Saved plans are stored in browser localStorage because the current project does not include authentication or user accounts. This is appropriate for the project scope, but it means saved plans are local to the browser.

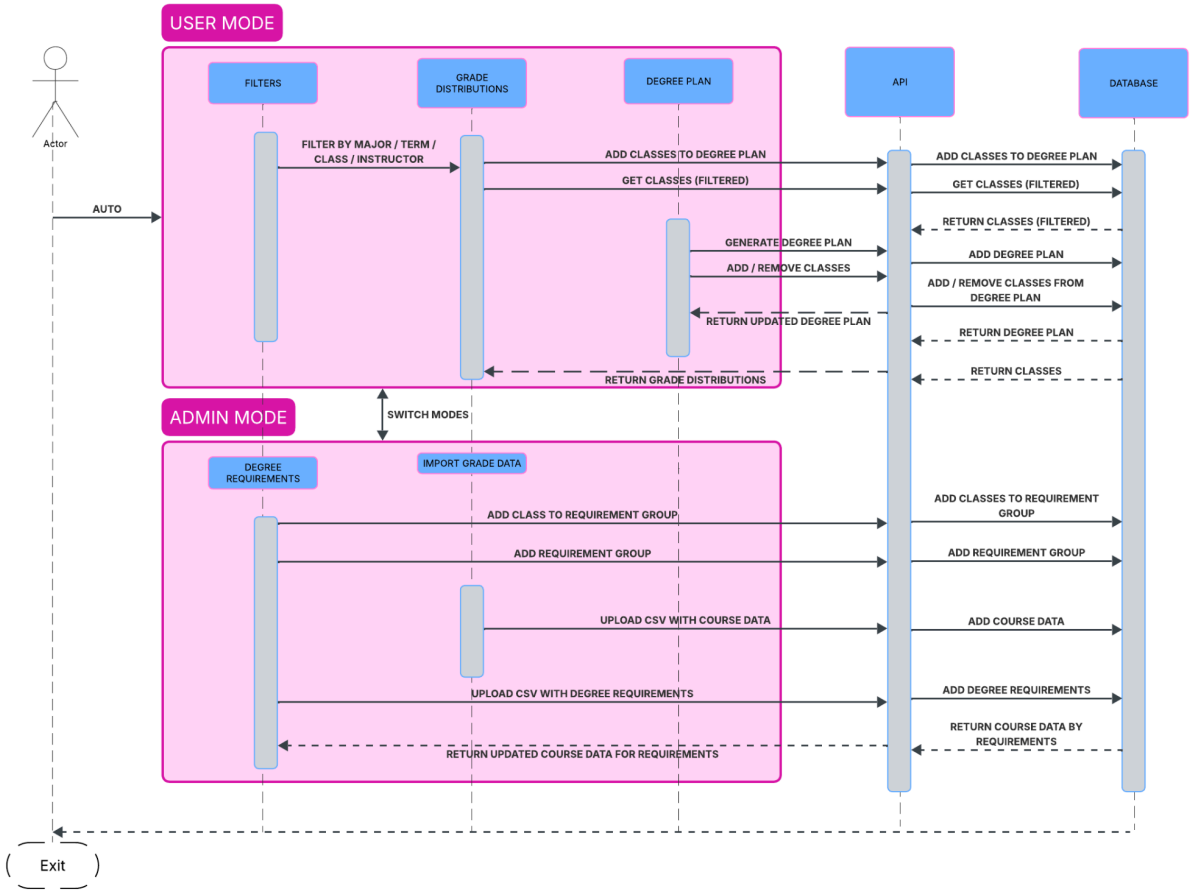
PDF export is handled in the browser because the saved plan is already visually rendered on the saved plans page. Capturing the rendered plan and exporting it as a PDF avoids building a separate backend report-generation system.

[9] Get Courses Request Sequence

Dynamic Models of Operational Scenarios

The following diagram summarizes the major operational scenarios of the system. These may overlap with the module-level dynamic models above, but they show the complete user-facing workflows.

Dynamic User Model



10

Figure 10 combines the major operational scenarios into a single dynamic model. The upper section shows student mode, where the user filters courses, views grade distributions, and generates or updates a degree plan. The lower section shows admin mode, where an admin uploads course data, uploads degree requirement data, and edits requirement groups. Both modes communicate through the API layer, which retrieves from or writes to the database layer.

[10] Combined dynamic model of MaxGPA operational scenarios, showing student mode workflows for filtering courses, viewing grade distributions, and generating degree plans, as well as admin mode workflows for importing grade data and editing degree requirements.

Alternative Designs

PostgreSQL Instead of SQLite

The team considered using PostgreSQL instead of SQLite. PostgreSQL would provide stronger support for deployment and concurrent users, but it would require a separate database server and more setup. SQLite was chosen because the project uses a local dataset and can run with fewer external dependencies.

Server-Side Saved Plans Instead of Browser localStorage

The team considered storing saved plans in the database. This would allow saved plans to follow users across devices, but it would require login, user accounts, and additional API endpoints. Browser localStorage was chosen because it supports saved plans without adding authentication to the project scope.

Backend-Generated Chart Images Instead of Frontend Charts\

The team considered generating chart images on the backend. This was rejected because the frontend already has the data needed to render charts. Rendering charts with Recharts in the browser avoids managing temporary image files and makes charts update immediately when filters change.

Direct CSV Querying Instead of Database Import

The team considered reading directly from CSV files for each query. This would reduce database setup, but it would make filtering and aggregation slower and harder to maintain. Importing CSV data into SQLite provides a consistent schema and supports indexed queries.

References

- CIS 422 Software Design Specification Template (https://classes.cs.uoregon.edu/26S/cs422/Templates.html)
- Group 7 project feedback comments
- MaxGPA project source code (https://github.com/hnakae/Project-1-MaxGPA)
- Next.js documentation (https://nextjs.org/docs)
- FastAPI documentation (https://fastapi.tiangolo.com/)
- SQLite documentation (https://sqlite.org/docs.html)
- pandas documentation (https://pandas.pydata.org/docs/)
- Recharts documentation (https://recharts.github.io/)
- jsPDF documentation (https://artskydj.github.io/jsPDF/docs/jsPDF.html)
- html-to-image documentation (https://www.npmjs.com/package/html-to-image)

Acknowledgements

This Software Design Specification was prepared for Group 7's MaxGPA project in CIS 422 Software Methodologies. The document reflects the implemented project structure, including the Next.js frontend, FastAPI backend, SQLite database, CSV import utilities, requirement evaluation logic, browser-side saved plans, and PDF export functionality.